

FSD - Lecture 1 Notes

DOUBT LINK :

https://docs.google.com/forms/d/e/1FAIpQLSdf3pAwuk8oGmEw7gl9PCk1nsdb2Dwd3A0_vnefuWvd_5vPxQ/viewform?usp=publish-editor

1. The Evolution of the Web

The transition from 1.0 to 3.0 is characterized by a shift from consumption to creation, and finally to ownership.

Feature	Web 1.0 (The Static Web)	Web 2.0 (The Social Web)	Web 3.0 (The Semantic Web)
Era	1990 – 2005	2005 – Present	Future / Emerging
Role of User	Passive Reader	Content Creator (Social Media)	Owner (Decentralized)
Content	Static HTML pages	Dynamic, User-Generated	AI-driven, Blockchain-based
Data Control	Company-owned	Centralized (Big Tech)	Decentralized (Edge Computing)

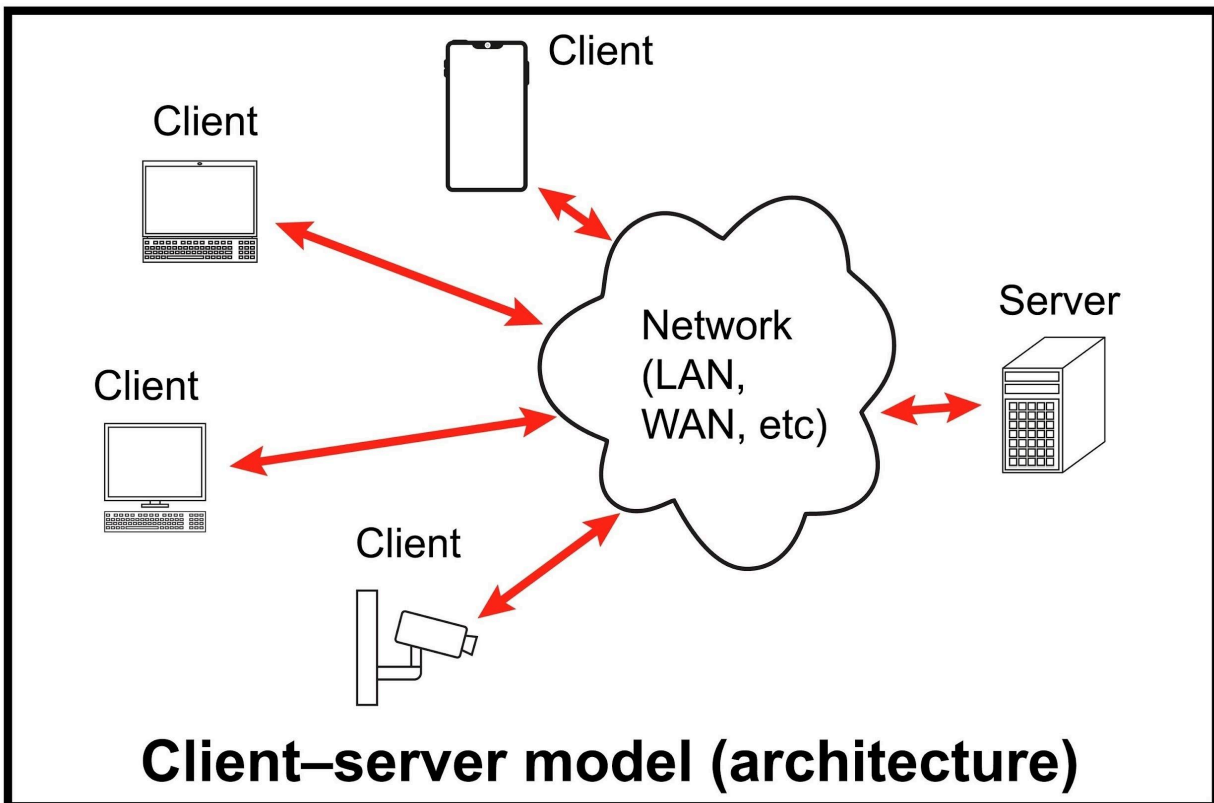
2. Architectures: Client-Server vs. 3-Tier

These terms are often used interchangeably, but they describe different levels of complexity.

The Client-Server Model

This is the fundamental "Request-Response" cycle.

- **Client:** The device/browser requesting data (e.g., Chrome on your laptop).
- **Server:** The machine that stores data and processes requests.



The 3-Tier Architecture

The 3-Tier model is a specific implementation of the Client-Server model that adds an extra layer of organization:

1. **Presentation Tier (Client):** The User Interface (HTML/CSS/JS).
2. **Application Tier (Server):** The logic/backend (Node.js, Python, Java). This processes data.
3. **Data Tier (Database):** Where information is stored (MongoDB, PostgreSQL).

The Relation: The Client-Server model is the "What" (two entities talking), while the 3-Tier architecture is the "How" (splitting the server's job into logic and storage).

3. Rendering: SSR vs. CSR

Rendering is the process of turning code (HTML/JS) into a viewable webpage.

Server-Side Rendering (SSR) - *Example: Django*

In SSR, the server does the heavy lifting. When you request a page, the server fetches data, populates an HTML template, and sends a **fully formed HTML page** to the browser.

- **Workflow:** User requests `/profile` $\rightarrow$ Django queries DB $\rightarrow$ Django renders HTML $\rightarrow$ Browser displays HTML immediately.
- **Pros:** Better SEO (search engines see content immediately), faster "First Contentful Paint."

Client-Side Rendering (CSR) - *Example: MERN*

In CSR, the server sends a "blank" HTML shell and a large JavaScript bundle. The browser then executes the JS to build the UI and fetch data.

- **Workflow:** User requests `/profile` $\rightarrow$ Express sends empty HTML + React JS $\rightarrow$ React runs in browser $\rightarrow$ React fetches data from API $\rightarrow$ Page populates.
 - **Pros:** Richer user interactions, feels like a mobile app (no full-page reloads).
-

4. Website Workflow Comparison

MERN (CSR Workflow)

1. **React (Client):** The user clicks a button. React handles the UI change.
2. **API Call:** React sends a JSON request to the Express server using `fetch` or `axios`.
3. **Express/Node (Server):** Receives the request, checks logic, and talks to MongoDB.
4. **MongoDB (Database):** Returns raw data (JSON) to the server.
5. **Data Return:** The server sends **only JSON** back to the browser.
6. **Browser Update:** React takes that JSON and updates the specific part of the page without refreshing.

Django (SSR Workflow)

1. **Browser:** User clicks a link.
2. **Django URL/View (Server):** Receives the request and pulls data from the Database (e.g., PostgreSQL).
3. **Template Engine:** Django injects that data into a `.html` file (using Django Template Language).
4. **Full Response:** Django sends the **entire HTML document** back to the browser.
5. **Browser Update:** The browser clears the screen and renders the new page from scratch.

Summary Table: CSR vs. SSR

Feature	CSR (MERN)	SSR (Django)
Initial Load	Slower (must download JS)	Faster (HTML ready to go)
Subsequent Navigation	Instant (only data changes)	Slower (reloads whole page)
Search Engine Optimization	Difficult (requires extra tools)	Excellent (native)

Server Load	Low (Client does the work)	High (Server renders every page)
--------------------	----------------------------	----------------------------------

How an API Request Works (The Cycle)

When your MERN app needs to display a list of users, it follows this "Request-Response" pattern:

- The Request (Client → Server):**
 - **Endpoint:** A URL like `https://api.myapp.com/users`
 - **Method:** `GET` (to fetch), `POST` (to create), `PUT` (to update), or `DELETE`.
 - **Headers:** Extra info, like an "Auth Token" to prove you're logged in.
- The Processing (Server):**
 - The API receives the request, validates the token, and asks the database for the user list.
- The Response (Server → Client):**
 - **Status Code:** `200 OK` (success), `404` (not found), or `500` (server error).
 - **Body (JSON):** ```json { "id": 101, "username": "DevUser", "email": "dev@example.com" }`

Frontend Optimization (Caching & CDNs)

When a browser tries to load a frontend asset (like `logo.png` or `app.js`), it follows this path:

- Browser Cache (Local):** "Do I already have this file on my hard drive?"
 - *If yes:* Load instantly. (**Cache Hit**)
- CDN Edge Server (Nearby):** "If not, is there a copy at a nearby data center in my city?"
 - *If yes:* The CDN sends it. This is much faster because the data doesn't travel across oceans.
- Origin Server (The Source):** "If the CDN doesn't have it, go to the actual server (where the MERN or Django app lives)."
 - *Action:* The CDN fetches it from the origin, **saves a copy for the next person**, and gives it to you.